

Oracle HR Schema Subquery Analytics & Business Reporting

1. Employees earning more than the company average salary

```
SELECT employee_id,  
       first_name || ' ' || last_name AS employee_name,  
       salary  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
);
```

Logic:

First, the subquery calculates the **average salary of all employees**.

Then, the main query displays employees whose salary is greater than that average.

Subquery type:

Scalar subquery because it returns **one value**: the company average salary.

Worksheet		Query Builder																																																																																																					
		<pre> SELECT employee_id, first_name ' ' last_name AS employee_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees); </pre>																																																																																																					
		Script Output x Query Result x SQL Fetched 50 rows in 0.012 seconds <table> <thead> <tr> <th></th><th>EMPLOYEE_ID</th><th>EMPLOYEE_NAME</th><th>SALARY</th></tr> </thead> <tbody> <tr><td>1</td><td>201</td><td>Michael Hartstein</td><td>13000</td></tr> <tr><td>2</td><td>203</td><td>Susan Mavris</td><td>6500</td></tr> <tr><td>3</td><td>204</td><td>Hermann Baer</td><td>10000</td></tr> <tr><td>4</td><td>205</td><td>Shelley Higgins</td><td>12008</td></tr> <tr><td>5</td><td>206</td><td>William Gietz</td><td>8300</td></tr> <tr><td>6</td><td>100</td><td>Steven King</td><td>24000</td></tr> <tr><td>7</td><td>101</td><td>Neena Kochhar</td><td>17000</td></tr> <tr><td>8</td><td>102</td><td>Lex De Haan</td><td>17000</td></tr> <tr><td>9</td><td>103</td><td>Alexander Hunold</td><td>9000</td></tr> <tr><td>10</td><td>108</td><td>Nancy Greenberg</td><td>12008</td></tr> <tr><td>11</td><td>109</td><td>Daniel Faviet</td><td>9000</td></tr> <tr><td>12</td><td>110</td><td>John Chen</td><td>8200</td></tr> <tr><td>13</td><td>111</td><td>Ismael Sciarra</td><td>7700</td></tr> <tr><td>14</td><td>112</td><td>Jose Manuel Urman</td><td>7800</td></tr> <tr><td>15</td><td>113</td><td>Luis Popp</td><td>6900</td></tr> <tr><td>16</td><td>114</td><td>Den Raphaely</td><td>11000</td></tr> <tr><td>17</td><td>120</td><td>Matthew Weiss</td><td>8000</td></tr> <tr><td>18</td><td>121</td><td>Adam Fripp</td><td>8200</td></tr> <tr><td>19</td><td>122</td><td>Payam Kaufling</td><td>7900</td></tr> <tr><td>20</td><td>123</td><td>Shanta Vollman</td><td>6500</td></tr> <tr><td>21</td><td>145</td><td>John Russell</td><td>14000</td></tr> <tr><td>22</td><td>146</td><td>Karen Partners</td><td>13500</td></tr> <tr><td>23</td><td>147</td><td>Alberto Errazuriz</td><td>12000</td></tr> <tr><td>24</td><td>148</td><td>Gerald Cambrault</td><td>11000</td></tr> </tbody> </table>			EMPLOYEE_ID	EMPLOYEE_NAME	SALARY	1	201	Michael Hartstein	13000	2	203	Susan Mavris	6500	3	204	Hermann Baer	10000	4	205	Shelley Higgins	12008	5	206	William Gietz	8300	6	100	Steven King	24000	7	101	Neena Kochhar	17000	8	102	Lex De Haan	17000	9	103	Alexander Hunold	9000	10	108	Nancy Greenberg	12008	11	109	Daniel Faviet	9000	12	110	John Chen	8200	13	111	Ismael Sciarra	7700	14	112	Jose Manuel Urman	7800	15	113	Luis Popp	6900	16	114	Den Raphaely	11000	17	120	Matthew Weiss	8000	18	121	Adam Fripp	8200	19	122	Payam Kaufling	7900	20	123	Shanta Vollman	6500	21	145	John Russell	14000	22	146	Karen Partners	13500	23	147	Alberto Errazuriz	12000	24	148	Gerald Cambrault	11000
	EMPLOYEE_ID	EMPLOYEE_NAME	SALARY																																																																																																				
1	201	Michael Hartstein	13000																																																																																																				
2	203	Susan Mavris	6500																																																																																																				
3	204	Hermann Baer	10000																																																																																																				
4	205	Shelley Higgins	12008																																																																																																				
5	206	William Gietz	8300																																																																																																				
6	100	Steven King	24000																																																																																																				
7	101	Neena Kochhar	17000																																																																																																				
8	102	Lex De Haan	17000																																																																																																				
9	103	Alexander Hunold	9000																																																																																																				
10	108	Nancy Greenberg	12008																																																																																																				
11	109	Daniel Faviet	9000																																																																																																				
12	110	John Chen	8200																																																																																																				
13	111	Ismael Sciarra	7700																																																																																																				
14	112	Jose Manuel Urman	7800																																																																																																				
15	113	Luis Popp	6900																																																																																																				
16	114	Den Raphaely	11000																																																																																																				
17	120	Matthew Weiss	8000																																																																																																				
18	121	Adam Fripp	8200																																																																																																				
19	122	Payam Kaufling	7900																																																																																																				
20	123	Shanta Vollman	6500																																																																																																				
21	145	John Russell	14000																																																																																																				
22	146	Karen Partners	13500																																																																																																				
23	147	Alberto Errazuriz	12000																																																																																																				
24	148	Gerald Cambrault	11000																																																																																																				

2. Employees earning less than the company's highest salary

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,

```

```
    salary
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Logic:

The subquery finds the **highest salary in the company**.

The main query returns employees who earn less than that highest salary.

Subquery type:

Scalar subquery because it returns **one value**: the maximum salary.

Worksheet		Query Builder	
		<pre> SELECT employee_id, first_name ' ' last_name AS employee_name, salary FROM employees WHERE salary < (SELECT MAX(salary) FROM employees); </pre>	
Script Output x		Query Result x	
		SQL Fetched 50 rows in 0.006 seconds	
	EMPLOYEE_ID	EMPLOYEE_NAME	SALARY
1	198	Donald OConnell	2600
2	199	Douglas Grant	2600
3	200	Jennifer Whalen	4400
4	201	Michael Hartstein	13000
5	202	Pat Fay	6000
6	203	Susan Mavris	6500
7	204	Hermann Baer	10000
8	205	Shelley Higgins	12008
9	206	William Gietz	8300
10	101	Neena Kochhar	17000
11	102	Lex De Haan	17000
12	103	Alexander Hunold	9000
13	104	Bruce Ernst	6000
14	105	David Austin	4800
15	106	Valli Pataballa	4800
16	107	Diana Lorentz	4200
17	108	Nancy Greenberg	12008
18	109	Daniel Faviyet	9000
19	110	John Chen	8200
20	111	Ismael Sciarra	7700
21	112	Jose Manuel Urman	7800
22	113	Luis Popp	6900
23	114	Den Raphaely	11000
24	115	Alexander Khoo	3100

3. Departments whose average salary exceeds the company average salary

```

SELECT department_id,
       AVG(salary) AS department_avg_salary

```

```
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM employees
);
```

Logic:

The main query calculates the **average salary for each department**.

The subquery calculates the **company average salary**.

Only departments with an average salary higher than the company average are displayed.

Subquery type:

Scalar subquery used inside **HAVING**.

[illegible]

4. Employees earning the highest salary within their department

```
SELECT employee_id,
```

```
    first_name || ' ' || last_name AS employee_name,  
    department_id,  
    salary  
FROM employees e  
WHERE salary = (  
    SELECT MAX(salary)  
    FROM employees  
    WHERE department_id = e.department_id  
);
```

Logic:

For each employee, the subquery checks the **highest salary in that employee's department**.

If the employee's salary equals that department maximum, the employee is displayed.

Subquery type:

Correlated subquery because the subquery depends on the outer query using:

e.department_id

Worksheet

Query Builder

<

1. Employees working in departments located in the United States

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,
       department_id,
       salary
FROM employees
WHERE department_id IN (
    SELECT department_id
    FROM departments
    WHERE location_id IN (
        SELECT location_id

```

```
        FROM locations
        WHERE country_id = 'US'
    )
);
```

Logic:

First, Oracle finds locations where `country_id = 'US'`.
Then it finds departments in those locations.
Finally, it displays employees working in those departments.

Subquery type:

Nested multi-row subquery using `IN`.

Worksheet

Query Builder

SELECT

employee_id,

first_name || ' ' || last_name AS employee_name,

department_id,

salary

FROM employees

WHERE department_id IN (

SELECT

department_id

FROM departments

WHERE location_id IN (

SELECT

location_id

FROM locations

WHERE country_id = 'US'

)

);

Script Output x

Query Result x

SQL

Fetched 50 rows in 0.013 seconds

	EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT_ID	SALARY
1	103	Alexander Hunold	60	9000
2	104	Bruce Ernst	60	6000
3	105	David Austin	60	4800
4	106	Valli Pataballa	60	4800
5	107	Diana Lorentz	60	4200
6	198	Donald OConnell	50	2600
7	199	Douglas Grant	50	2600
8	120	Matthew Weiss	50	8000
9	121	Adam Fripp	50	8200
10	122	Payam Kaufling	50	7900
11	123	Shanta Vollman	50	6500
12	124	Kevin Mourgous	50	5800
13	125	Julia Nayer	50	3200
14	126	Irene Mikkilineni	50	2700
15	127	James Landry	50	2400
16	128	Steven Markle	50	2200
17	129	Laura Bissot	50	3300
18	130	Mozhe Atkinson	50	2800
19	131	James Marlow	50	2500
20	132	TJ Olson	50	2100

2. Departments that currently contain employees

```

SELECT department_id,
       department_name
FROM departments
WHERE department_id IN (

```

```
SELECT DISTINCT department_id
FROM employees
WHERE department_id IS NOT NULL
);
```

Logic:

The subquery gets all department IDs that are used by employees.

The main query displays only departments that have at least one employee.

Subquery type:

Multi-row subquery using **IN**.

The screenshot shows a database interface with a 'Query Builder' tab. The SQL query entered is:

```
SELECT department_id,
       department_name
FROM departments
WHERE department_id IN (
    SELECT DISTINCT department_id
    FROM employees
    WHERE department_id IS NOT NULL
);
```

Below the query editor, the 'Query Result' tab is active, showing the results of the query. The results are displayed in a table with two columns: DEPARTMENT_ID and DEPARTMENT_NAME. The table contains 11 rows of data.

	DEPARTMENT_ID	DEPARTMENT_NAME
1	10	Administration
2	20	Marketing
3	30	Purchasing
4	40	Human Resources
5	50	Shipping
6	60	IT
7	70	Public Relations
8	80	Sales
9	90	Executive
10	100	Finance
11	110	Accounting

3. Departments that currently have no employees

```
SELECT department_id,  
       department_name  
FROM departments d  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM employees e  
    WHERE e.department_id = d.department_id  
);
```

Logic:

For each department, Oracle checks if there is any employee assigned to it.
If no employee exists, the department is displayed.

Subquery type:

Correlated subquery using **NOT EXISTS**.

Note:

NOT EXISTS is safer than **NOT IN** because `employees.department_id` may contain **NULL**.


```
        salary
FROM employees
WHERE department_id IN (
    SELECT department_id
    FROM departments
    WHERE location_id IN (
        SELECT location_id
        FROM locations
        WHERE country_id IN (
            SELECT country_id
            FROM countries
            WHERE region_id IN (
                SELECT region_id
                FROM regions
                WHERE region_name = 'Europe'
            )
        )
    )
);
```

Logic:

Oracle starts from the innermost subquery:

1. Finds the **region_id** for Europe.
2. Finds countries in Europe.
3. Finds locations in those countries.
4. Finds departments in those locations.
5. Displays employees working in those departments.

Subquery type:

Nested multi-row subquery.


```

        salary,
        manager_id
FROM employees e
WHERE salary > (
    SELECT salary
    FROM employees m
    WHERE m.employee_id = e.manager_id
);

```

Logic:

For each employee, the subquery finds the salary of that employee's manager. Then the main query displays only employees whose salary is higher than their manager's salary.

Subquery type:

Correlated scalar subquery.

The screenshot shows a database query builder interface. The top tab is 'Query Builder'. The SQL query is as follows:

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,
       salary,
       manager_id
FROM employees e
WHERE salary > (
    SELECT salary
    FROM employees m
    WHERE m.employee_id = e.manager_id
);

```

The bottom tab is 'Query Result'. It shows the results of the query, indicating that all rows were fetched in 0.005 seconds. The results are displayed in a table with the following columns: EMPLOYEE_ID, EMPLOYEE_NAME, SALARY, and MANAGER_ID.

EMPLOYEE_ID	EMPLOYEE_NAME	SALARY	MANAGER_ID
1	168 Lisa Ozer	11500	148
2	174 Ellen Abel	11000	149

2. Managers supervising at least one employee earning more than 10,000

```

SELECT employee_id,
       first_name || ' ' || last_name AS manager_name,

```

```

        salary
FROM employees m
WHERE employee_id IN (
    SELECT manager_id
    FROM employees
    WHERE salary > 10000
    AND manager_id IS NOT NULL
);

```

Logic:

The subquery finds manager IDs of employees who earn more than 10,000.
The main query displays those managers.

Subquery type:

Multi-row subquery using **IN**.

The screenshot shows a SQL Query Builder interface. The top pane displays the following SQL query:

```

SELECT employee_id,
       first_name || ' ' || last_name AS manager_name,
       salary
FROM employees m
WHERE employee_id IN (
    SELECT manager_id
    FROM employees
    WHERE salary > 10000
    AND manager_id IS NOT NULL
);

```

The bottom pane shows the query results in a table format. The status bar indicates "All Rows Fetched: 5 in 0.005 seconds".

	EMPLOYEE_ID	MANAGER_NAME	SALARY
1	100	Steven King	24000
2	101	Neena Kochhar	17000
3	147	Alberto Errazuriz	12000
4	148	Gerald Cambrault	11000
5	149	Eleni Zlotkey	10500

3. Employees whose manager works in the same department

```
SELECT employee_id,  
       first_name || ' ' || last_name AS employee_name,  
       department_id,  
       manager_id  
FROM employees e  
WHERE department_id = (  
    SELECT department_id  
    FROM employees m  
    WHERE m.employee_id = e.manager_id  
);
```

Logic:

For each employee, the subquery finds the department of the employee's manager.
The main query displays employees whose department is the same as their manager's department.

Subquery type:

Correlated scalar subquery.

Worksheet

Query Builder

4. Employees whose manager works in a different department

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,
       department_id,

```

```
        manager_id
FROM employees e
WHERE department_id <> (
    SELECT department_id
    FROM employees m
    WHERE m.employee_id = e.manager_id
);
```

Logic:

For each employee, Oracle checks the manager's department.

If the employee's department is different from the manager's department, the employee is displayed.

Subquery type:

Correlated scalar subquery.

Worksheet

Query Builder

1. Filtering employees earning more than 10,000

```

SELECT employee_id,
       employee_name,
       salary,
       department_id

```

```
FROM (  
  SELECT employee_id,  
         first_name || ' ' || last_name AS employee_name,  
         salary,  
         department_id  
  FROM employees  
) emp_view  
WHERE salary > 10000;
```

Logic:

The inline view first creates a temporary result containing employee details.

The outer query filters that result and displays only employees earning more than 10,000.

Subquery type:

Inline view because the subquery is inside the FROM clause.

Worksheet

Query Builder

```

SELECT employee_id,
       employee_name,
       salary,
       department_id
FROM (
    SELECT employee_id,
           first_name || ' ' || last_name AS employee_name,
           salary,
           department_id
    FROM employees
) emp_view
WHERE salary > 10000;

```

Script Output x

Query Result x

SQL | All Rows Fetched: 15 in 0.003 seconds

	EMPLOYEE_ID	EMPLOYEE_NAME	SALARY	DEPARTMENT_ID
1	201	Michael Hartstein	13000	20
2	205	Shelley Higgins	12008	110
3	100	Steven King	24000	90
4	101	Neena Kochhar	17000	90
5	102	Lex De Haan	17000	90
6	108	Nancy Greenberg	12008	100
7	114	Den Raphaely	11000	30
8	145	John Russell	14000	80
9	146	Karen Partners	13500	80
10	147	Alberto Errazuriz	12000	80
11	148	Gerald Cambrault	11000	80
12	149	Eleni Zlotkey	10500	80
13	162	Clara Vishney	10500	80
14	168	Lisa Ozer	11500	80
15	174	Ellen Abel	11000	80

2. Displaying departments with average salary greater than 9,000

```

SELECT department_id,
       avg_salary
FROM (
    SELECT department_id,
           AVG(salary) AS avg_salary
    FROM employees

```

) dept_avg_view

The inline view calculates the average salary for each department.

The outer query filters only departments where the average salary is greater than 9,000.

Inline view with aggregation.

Inline view with aggregation.

[illegible]

employees

FROM (

```

SELECT department_id,
       COUNT(*) AS employee_count
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
) dept_count_view
WHERE employee_count > 5;

```

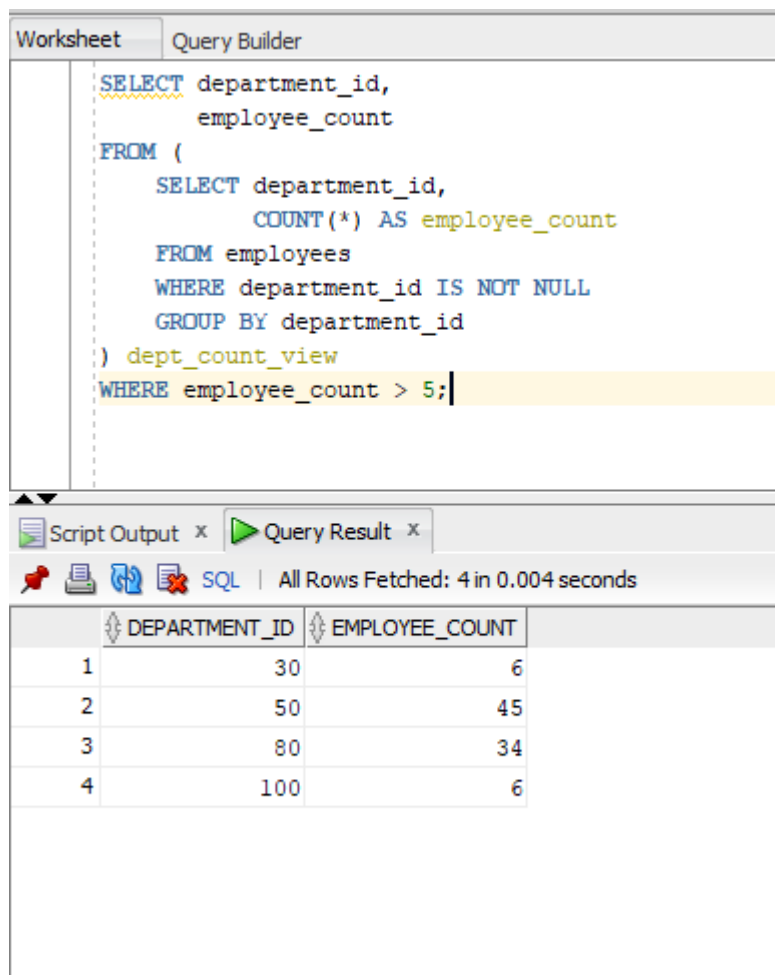
Logic:

The inline view counts how many employees are in each department.

The outer query displays only departments that have more than five employees.

Subquery type:

Inline view with **GROUP BY** and **COUNT**.



The screenshot shows a SQL Query Builder window with a query defined in an inline view. The query selects department_id and employee_count from employees, grouped by department_id, and then filters for departments with more than 5 employees. The results are displayed in a table with 4 rows.

```

SELECT department_id,
       employee_count
FROM (
    SELECT department_id,
           COUNT(*) AS employee_count
    FROM employees
    WHERE department_id IS NOT NULL
    GROUP BY department_id
) dept_count_view
WHERE employee_count > 5;

```

	DEPARTMENT_ID	EMPLOYEE_COUNT
1	30	6
2	50	45
3	80	34
4	100	6

1. Employees earning above their department average salary

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,

```

```
    department_id,  
    salary  
FROM employees e  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE department_id = e.department_id  
);
```

Logic:

For each employee, the subquery calculates the average salary of that employee's department.

Then the main query displays employees whose salary is above their department average.

Subquery type:

Correlated scalar subquery.

Worksheet		Query Builder		
		<pre> SELECT employee_id, first_name ' ' last_name AS employee_name, department_id, salary FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id); </pre>		
		Script Output x Query Result x SQL All Rows Fetched: 38 in 0.01 seconds		
	EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT_ID	SALARY
1	201	Michael Hartstein	20	13000
2	205	Shelley Higgins	110	12008
3	100	Steven King	90	24000
4	103	Alexander Hunold	60	9000
5	104	Bruce Ernst	60	6000
6	108	Nancy Greenberg	100	12008
7	109	Daniel Faviet	100	9000
8	114	Den Raphaely	30	11000
9	120	Matthew Weiss	50	8000
10	121	Adam Fripp	50	8200
11	122	Payam Kaufling	50	7900
12	123	Shanta Vollman	50	6500
13	124	Kevin Mourgous	50	5800
14	137	Renske Ladwig	50	3600
15	141	Trenna Rajs	50	3500
16	145	John Russell	80	14000
17	146	Karen Partners	80	13500
18	147	Alberto Errazuriz	80	12000
19	148	Gerald Cambrault	80	11000
20	149	Eleni Zlotkey	80	10500
21	150	Peter Tucker	80	10000
22	151	David Bernstein	80	9500

2. Employees earning above the average salary for their job role

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,

```

```
    job_id,  
    salary  
FROM employees e  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE job_id = e.job_id  
);
```

Logic:

For each employee, the subquery calculates the average salary for the same job role.
The main query displays employees earning more than the average salary of their job.

Subquery type:

Correlated scalar subquery.

Worksheet

Query Builder

<

3. Departments where all employees earn more than 5,000

```

SELECT department_id,
       department_name

```

```
FROM departments d
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
)
AND NOT EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
    AND e.salary <= 5000
);
```

Logic:

The first **EXISTS** makes sure the department has employees.

The second **NOT EXISTS** checks that there is no employee in that department earning 5,000 or less.

So only departments where **all employees earn more than 5,000** are displayed.

Subquery type:

Correlated subquery using **EXISTS** and **NOT EXISTS**.

Worksheet

Query Builder

SELECT department_id,

department_name

FROM departments d

WHERE EXISTS (

SELECT 1

FROM employees e

WHERE e.department_id = d.department_id

)

AND NOT EXISTS (

SELECT 1

FROM employees e

WHERE e.department_id = d.department_id

AND e.salary <= 5000

);

Script Output x

Query Result x

SQL

All Rows Fetched: 7 in 0.004 seconds

	DEPARTMENT_ID	DEPARTMENT_NAME
1	20	Marketing
2	40	Human Resources
3	70	Public Relations
4	80	Sales
5	90	Executive
6	100	Finance
7	110	Accounting

4. Employees who are the only employee in their department

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,
       department_id,
       salary
FROM employees e
WHERE (
  SELECT COUNT(*)
  FROM employees
  WHERE department_id = e.department_id
) = 1;

```

Logic:

For each employee, the subquery counts how many employees are in the same department.

If the count equals 1, it means that employee is the only employee in that department.

Subquery type:

Correlated scalar subquery.

The screenshot shows a SQL Query Builder window with two tabs: 'Worksheet' and 'Query Builder'. The 'Query Builder' tab is active, displaying the following SQL query:

```
SELECT employee_id,  
       first_name || ' ' || last_name AS employee_name,  
       department_id,  
       salary  
FROM employees e  
WHERE (  
    SELECT COUNT(*)  
    FROM employees  
    WHERE department_id = e.department_id  
) = 1;
```

Below the query editor, there is a 'Script Output' tab and a 'Query Result' tab. The 'Query Result' tab is active, showing the results of the query. The status bar indicates 'All Rows Fetched: 3 in 0.005 seconds'.

	EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT_ID	SALARY
1	200	Jennifer Whalen	10	4400
2	203	Susan Mavris	40	6500
3	204	Hermann Baer	70	10000

1. Employees earning above the company average salary

```
SELECT employee_id,  
       first_name || ' ' || last_name AS employee_name,  
       salary  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees
```

);

Logic:

The subquery calculates the average salary of the whole company.

The main query displays employees earning more than that average.

Subquery type:

Scalar subquery.


```
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
HAVING COUNT(*) > (
    SELECT COUNT(*)
    FROM employees
    WHERE department_id = 50
);
```

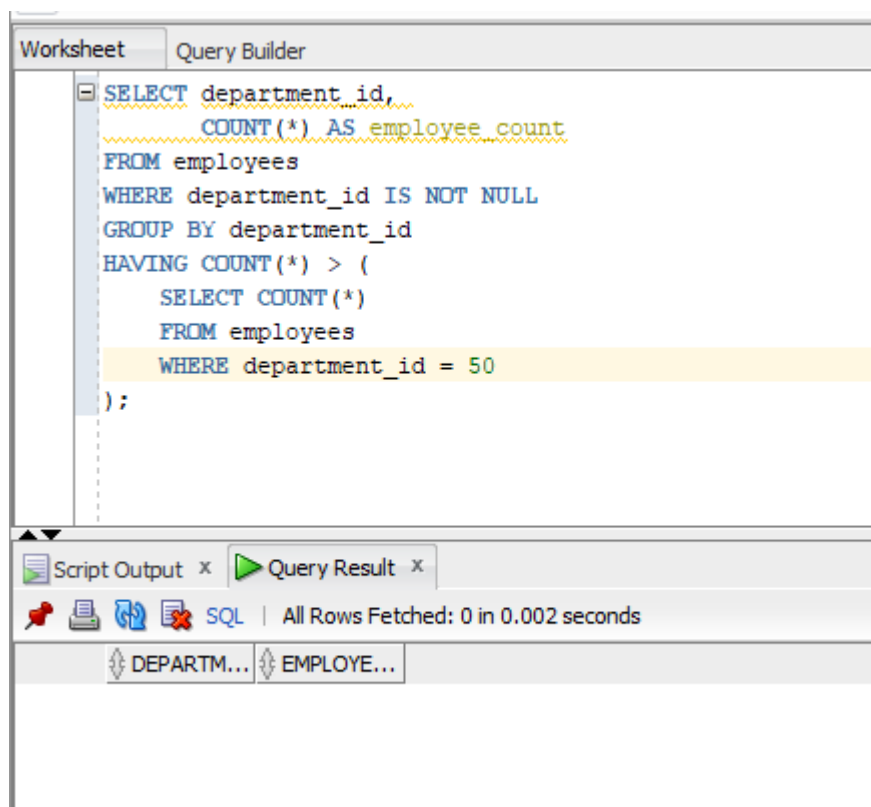
Logic:

The subquery counts how many employees are in Department 50.

The main query counts employees in each department and displays departments with a higher count.

Subquery type:

Scalar subquery inside **HAVING**.

**Result Explanation:**

The query returned no rows because Department 50 has the highest number of employees, with 45 employees. Therefore, no department contains more employees than Department 50.

Worksheet

Query Builder

```
SELECT department_id,
       COUNT(*) AS employee_count
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
ORDER BY employee_count DESC;
```

Script Output x

Query Result x

 SQL | All Rows Fetched: 11 in 0.004 seconds

	DEPARTMENT_ID	EMPLOYEE_COUNT
1	50	45
2	80	34
3	100	6
4	30	6
5	60	5
6	90	3
7	110	2
8	20	2
9	70	1
10	10	1
11	40	1

3. Employees working in the department with the highest average salary

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,
       department_id,
       salary
FROM employees
WHERE department_id IN (
  SELECT department_id
  FROM employees
  WHERE department_id IS NOT NULL
  GROUP BY department_id
  HAVING AVG(salary) = (

```

```
SELECT MAX(avg_salary)
FROM (
    SELECT AVG(salary) AS avg_salary
    FROM employees
    WHERE department_id IS NOT NULL
    GROUP BY department_id
)
);
```

Logic:

The inline view calculates the average salary for each department.
Then the nested subquery finds the highest department average salary.
The main query displays employees working in the department that has that highest average.

Subquery type:

Nested subquery with inline view.

Worksheet

Query Builder

SELECT employee_id,

first_name || ' ' || last_name AS employee_name,

department_id,

salary

FROM employees

WHERE department_id IN (

SELECT department_id

FROM employees

WHERE department_id IS NOT NULL

GROUP BY department_id

HAVING AVG(salary) = (

SELECT MAX(avg_salary)

FROM (

SELECT AVG(salary) AS avg_salary

FROM employees

WHERE department_id IS NOT NULL

GROUP BY department_id

Script Output x

Query Result x

SQL

All Rows Fetched: 3 in 0.008 seconds

	EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT_ID	SALARY
1	100	Steven King	90	24000
2	101	Neena Kochhar	90	17000
3	102	Lex De Haan	90	17000

4. Employees earning the second-highest salary

```

SELECT employee_id,
       first_name || ' ' || last_name AS employee_name,
       salary
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (
        SELECT MAX(salary)
        FROM employees
    )
);

```

Logic:

The innermost subquery finds the highest salary.

The outer subquery finds the maximum salary lower than the highest salary.

The main query displays employees earning that second-highest salary.

Subquery type:

Nested scalar subquery.

The screenshot shows a SQL Query Builder interface with a 'Query Builder' tab. The query is as follows:

```
SELECT employee_id,  
       first_name || ' ' || last_name AS employee_name,  
       salary  
FROM employees  
WHERE salary = (  
  SELECT MAX(salary)  
  FROM employees  
  WHERE salary < (  
    SELECT MAX(salary)  
    FROM employees  
  )  
);
```

Below the query editor, the 'Query Result' tab is active, showing the results of the query. The status bar indicates 'All Rows Fetched: 2 in 0.004 seconds'.

	EMPLOYEE_ID	EMPLOYEE_NAME	SALARY
1	101	Neena Kochhar	17000
2	102	Lex De Haan	17000

5. Managers whose employees' average salary exceeds 10,000

```
SELECT employee_id,  
       first_name || ' ' || last_name AS manager_name  
FROM employees m  
WHERE (  
  SELECT AVG(e.salary)  
  FROM employees e  
  WHERE e.manager_id = m.employee_id  
) > 10000;
```

Logic:

For each manager, the subquery calculates the average salary of employees supervised by that manager.

The main query displays managers whose employees' average salary is greater than 10,000.

Subquery type:

Correlated scalar subquery.

The screenshot shows a SQL Query Builder window with a worksheet tab. The query is as follows:

```
SELECT employee_id,  
       first_name || ' ' || last_name AS manager_name  
FROM employees m  
WHERE (  
    SELECT AVG(e.salary)  
    FROM employees e  
    WHERE e.manager_id = m.employee_id  
) > 10000;
```

Below the query editor, the 'Query Result' tab is active, showing the results of the query. The status bar indicates 'All Rows Fetched: 1 in 0.005 seconds'. The result is displayed in a table with two columns: EMPLOYEE_ID and MANAGER_NAME.

EMPLOYEE_ID	MANAGER_NAME
1	100 Steven King